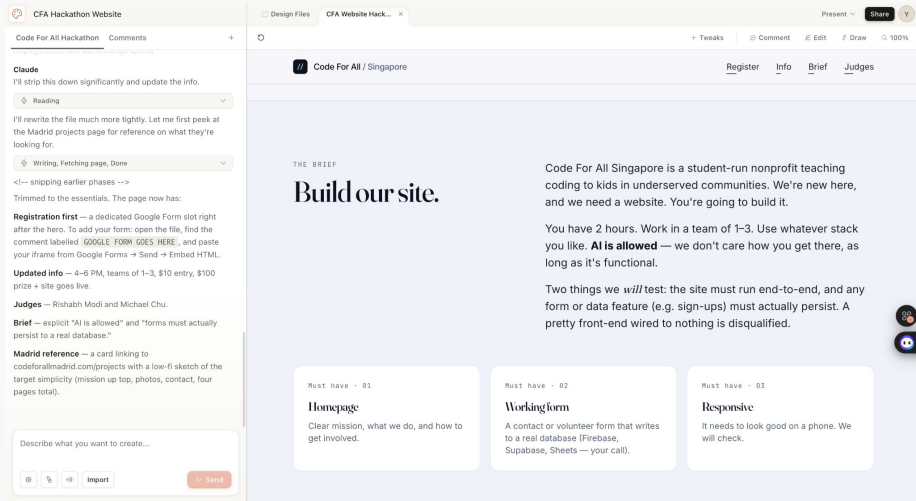
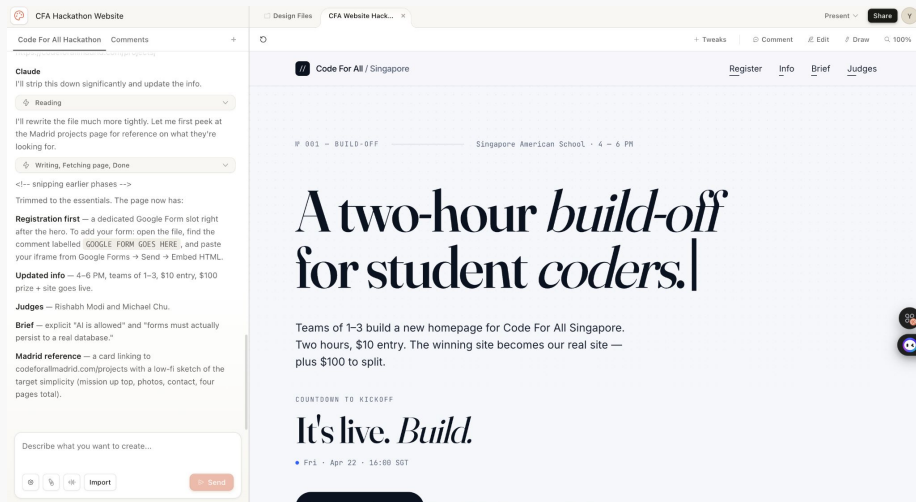

LaunchLab 5

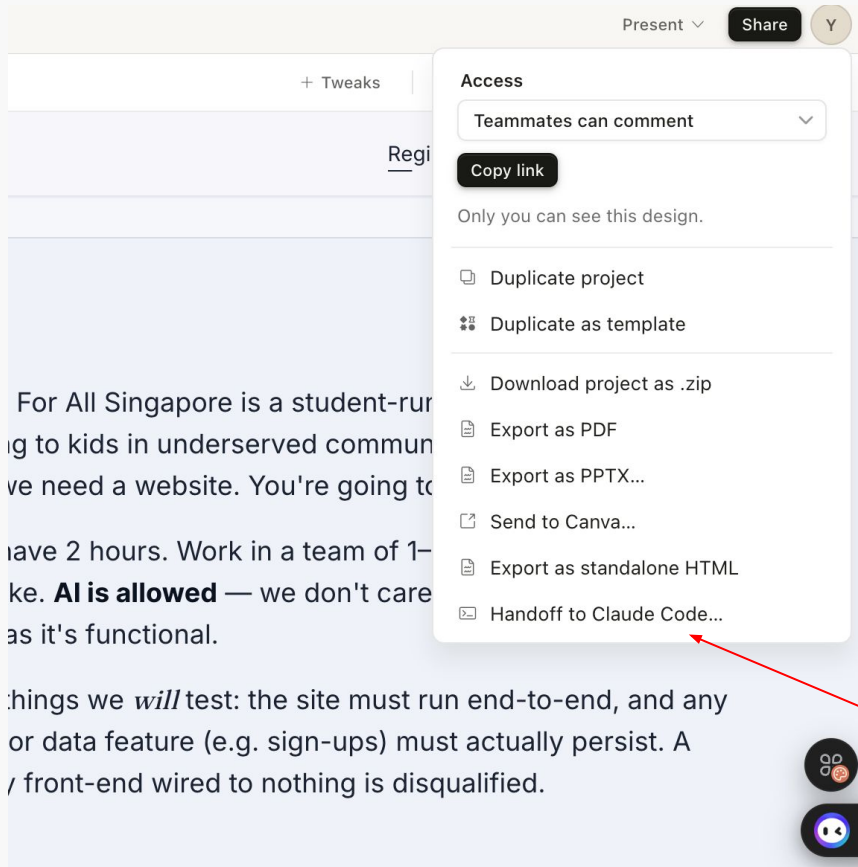
Lesson 5: Introduction to Claude Design

Design Your Front End First



- Claude Design allows you to build the visual shell of your site
- Describe how you want it to look without any code!

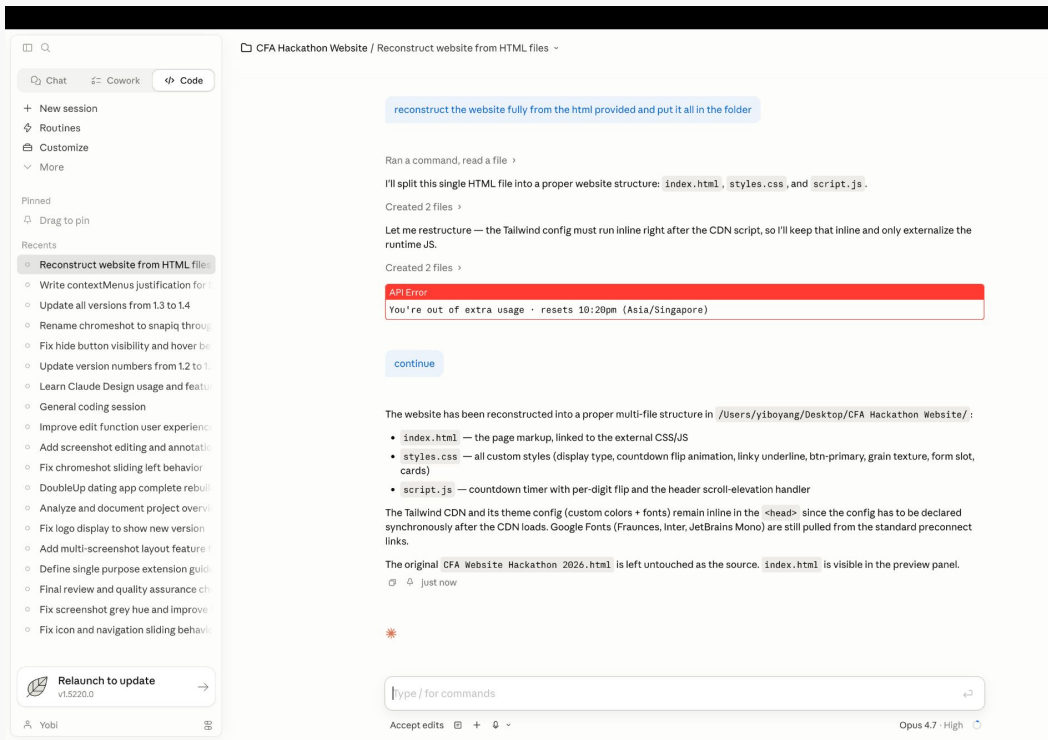
Export Your Design



- When your design is done, open the **Share** menu in the top right
- Find "**Handoff to Claude Code...**" in the export options — this is the bridge that connects the two tools
- It sends your finished design straight into Claude Code, which turns it into a real, working site
- Locate this button *before* you start building — you'll need it, and it's easy to miss in the menu

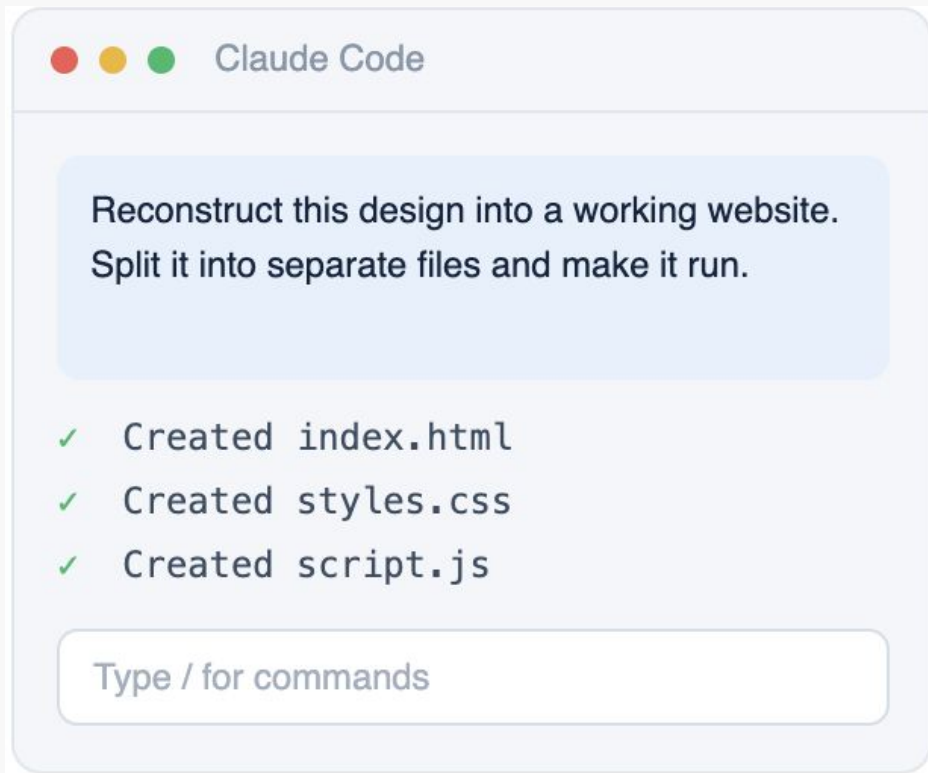
What Claude Code Actually Does

• • • • •



- Claude Code is an **agent**, not a chatbot — instead of just replying in chat, it works directly on the files on your computer
- It splits your single design into proper, separate files: `index.html`, `styles.css`, and `script.js`
- It works on its own — running commands, reading files, and deciding what to do next without you guiding each step
- Errors are normal and not a sign you broke something — if you hit an API limit error, just type "**continue**" and it picks up exactly where it left off

What to do After Hand Off



1. It waits for you

It won't build on its own - you type an instruction first.

2. Tell it to build

Example Prompt: "Reconstruct this design into a working website. Split it into index.html, styles.css and script.js, and make sure it actually runs."

3. Watch it work

Hit an error? Just type continue.

4. Make it actually work

"Make the sign-up form really save what people type."

When it breaks(And it will)



1. Read the red. Don't panic

An error is information, not failure. It names the file and the line where things went wrong.

2. Press f12

Chrome DevTools opens. Click the console tab, red text is your actual error. This is the first thing real engineers look at - now it's the first thing you look at

3. Read the red. Don't panic

Paste the exact error, then say what you clicked and what you expected to happen

4. Reload and check it yourself.

Refresh the page and test the exact thing that broke. If it's still broken, say so and go around again.

`index.html`

The words and the structure — every heading, button and paragraph people actually read.

`styles.css`

The look — colours, fonts, spacing, and how it changes on a phone versus a laptop.

`script.js`

The behaviour — what happens when somebody clicks, types, or hits submit.

Before you leave today, answer these out loud

1. Which file do you open to change the text on your main button?
2. Which file do you open to make that button orange?
3. Which file do you open to make it do something when clicked?
4. Point at one line you didn't write and say what it does.

The Whole Workflow

1. Design

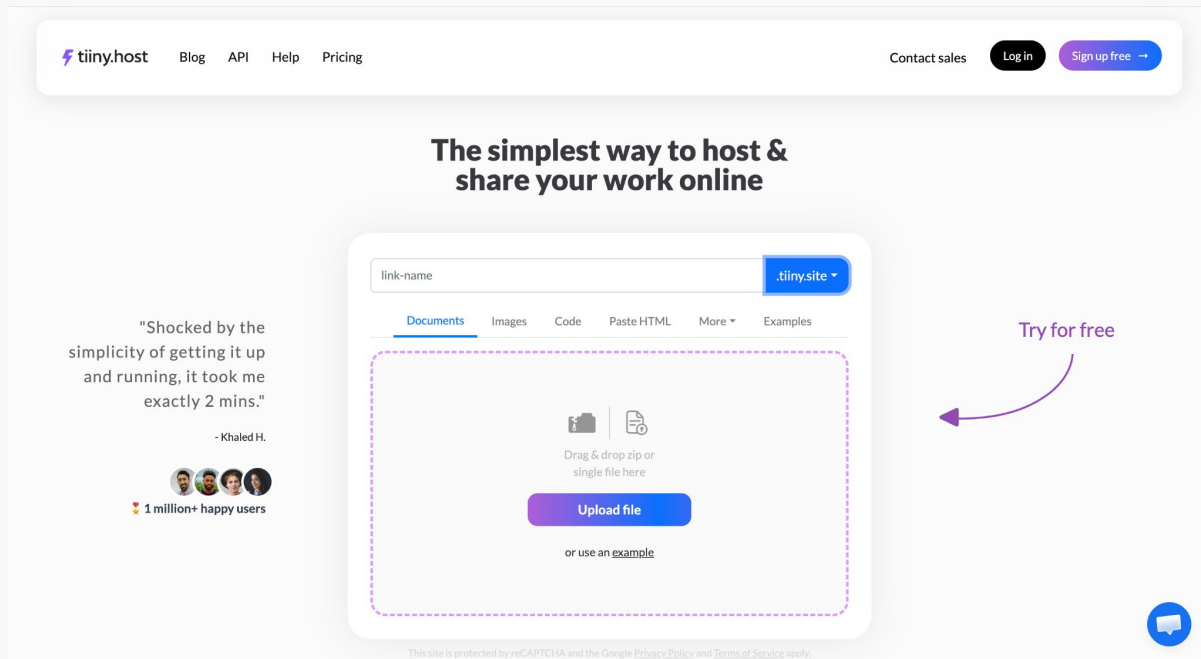
Use Claude Design to make your website look like how you want it to without any code.

2. Build

Use Claude Code to build the actual file by handing off the design you made in Claude Design.

3. Deploy

Use localhost or tiiny.host to launch your final website!

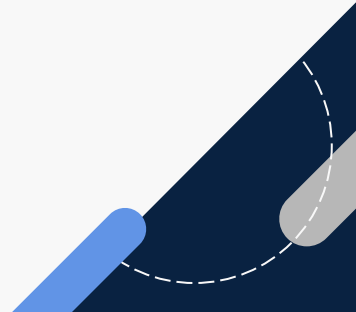


If using tiiny.host, put this prompt into Claude Code once you are done:

"Put all my website files into one folder and zip it, with index.html at the top level, so I can upload it to tiiny.host."



Your Turn!



Next Lesson: Introduction to GitHub and Vercel

Homework: Finish up creating your website.

Optional: Build a tool that automates something annoying in your school life. Any tool (lovable, claude, etc.) works. Examples: scrapes your school portal to text you when grades drop, generates a study plan from your test schedule, finds free classrooms based on the schedule.

DM your website link to Yibo, Rishabh, or Nirvan on Slack (use tiiny.host to publish or screen record it working on localhost)